Research 2
Template - Project plan 3
TrustX Adaptive API - REST API Documentation 5
Template - Decision documentation 8
Template - Meeting notes 9
Understanding AI Models for Privileged Access Management (PAM) & User Footprint Knowledge Graph 10
Developing Graph-Based Risk Analysis Using Graph Neural Networks (GNNs) and Neo4j 13
REST API vs. GraphQL for Frontend 16
TrustX Adaptive API - GraphQL Schema for Knowledge Graph 18

Research

Description

In a sentence or two, describe the purpose of this space.

Project Tracker

Recently updated content

This list below will automatically update each time somebody in your space creates or updates content.

TrustX Adaptive API - GraphQL Schema for Knowledge Graph
Feb 11, 2025 • contributed by Bhanu Nagarur

TrustX Adaptive API - REST API Documentation
Feb 11, 2025 • contributed by Bhanu Nagarur

REST API vs. GraphQL for Frontend
Feb 11, 2025 • contributed by Bhanu Nagarur

Developing Graph-Based Risk Analysis Using Graph Neural Networks (GNNs) and Neo4j
Feb 10, 2025 • contributed by Bhanu Nagarur

Understanding AI Models for Privileged Access Management (PAM) & User Footprint Knowledge Graph
Feb 10, 2025 • contributed by Bhanu Nagarur

Template - Meeting notes
Feb 10, 2025 • contributed by Bhanu Nagarur

Template - Decision documentation
Feb 10, 2025 • contributed by Bhanu Nagarur

Template - Project plan
Feb 10, 2025 • contributed by Bhanu Nagarur

Contributors

This list below will automatically update each time somebody in your space creates or updates content.

Bhanu Nagarur

😎 Template - Project plan

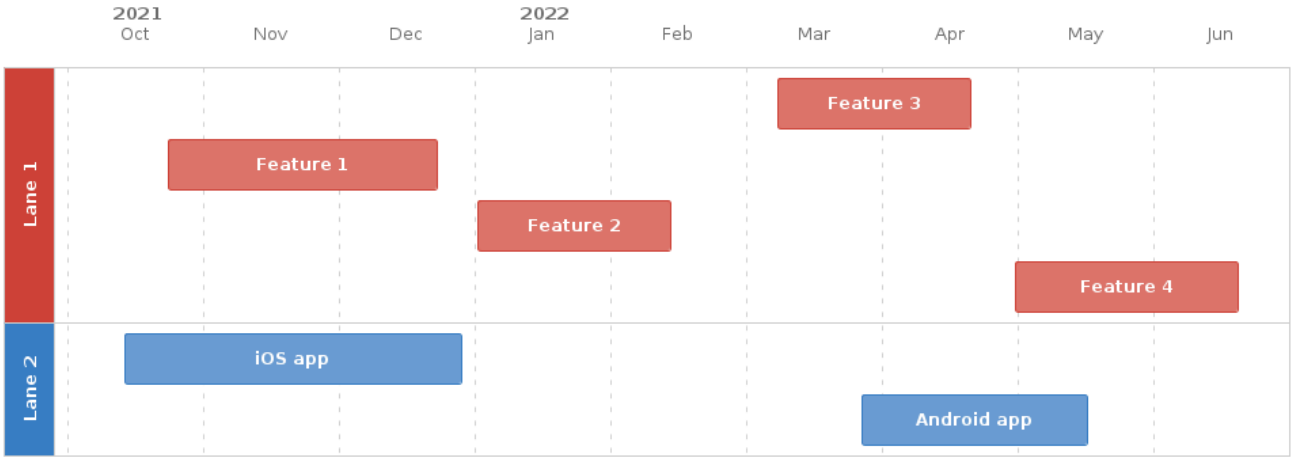
Driver	
Approver	
Contributors	
Informed	
Objective	
Due date	
Key outcomes	
Status	NOT STARTED / IN PROGRESS / COMPLETE

🤔 Problem Statement

🎯 Scope

Must have:	• •
Nice to have:	• •
Not in scope:	• •

🗓️ Timeline



Milestones and deadlines

Milestone	Owner	Deadline	Status

Reference materials

TrustX Adaptive API - REST API Documentation

1. Introduction

The **TrustX Adaptive API** provides RESTful endpoints for authentication, compliance tracking, integrations, and privileged access management. This API is designed to be **secure, scalable, and optimized for Cloudflare** caching.

Base URL

```
<https://api.trustx.cloud/>
```

---

2. Authentication & User Management

TrustX API uses **Auth0 (OAuth 2.0)** for authentication and **JWT-based authorization**.

Endpoints

Endpoint	Method	Description
`/auth/login`	`POST`	User authentication via Auth0
`/auth/logout`	`POST`	Ends user session
`/users/profile`	`GET`	Fetch authenticated user details

Example Request (Login)

```
POST /auth/login
Content-Type: application/json
{
  "username": "user@example.com",
  "password": "securepassword"
}
```

Response (Success)

```
{
  "access_token": "eyJhbGci...",
  "expires_in": 3600
}
```

---

3. Integration Management

Endpoints for **connecting, checking status, and removing integrations** like GitHub, AWS, and Google Workspace.

Endpoint	Method	Description
`/integrations/status`	`GET`	Fetch status of configured integrations

`/integrations/connect`	`POST`	Connect a third-party service
`/integrations/disconnect`	`DELETE`	Remove an existing integration

Example Response (Integration Status)

```
{
  "integration": "GitHub",
  "status": "active",
  "last_synced": "2025-02-11T12:00:00Z"
}
```

---

4. Compliance & Governance APIs

For tracking security controls, frameworks, and audit logs.

Endpoint	Method	Description
`/compliance/frameworks`	`GET`	Fetch available compliance frameworks
`/compliance/controls`	`GET`	Retrieve security controls mapped to a framework
`/audit/logs`	`GET`	Retrieve compliance audit logs

---

5. Risk & Security Assessment APIs

For detecting anomalies, assessing privileged access, and user behavior monitoring.

Endpoint	Method	Description
`/risk/assessments`	`GET`	Fetch risk assessment reports
`/risk/findings`	`GET`	Retrieve security vulnerabilities & compliance gaps

---

6. API Security Considerations

✅ **Authentication:** OAuth 2.0 (Auth0) with JWT tokens.
✅ **Rate Limiting:** API Gateway enforces request limits.
✅ **Logging & Monitoring:** AWS CloudWatch logs every API request.
✅ **Data Encryption:** TLS 1.2+ enforced.
✅ **Multi-Tenant Isolation:** Access is scoped per customer.

---

7. Next Steps

- Implement **GraphQL API** for dynamic queries.
- Document **hybrid API structure**.

- Set up **frontend integration guidelines**.

✅ Template - Decision documentation

Status	NOT STARTED / IN PROGRESS / COMPLETE
Impact	HIGH / MEDIUM / LOW
Driver	
Approver	
Contributors	
Informed	
Due date	
Resources	

📘 Background

🗂️ Relevant data

🌈 Options considered

	Option 1	Option 2
Description		
Pros and cons	➕ ➖	➕ ➖
Estimated cost	LARGE	MEDIUM

✅ Action items

- [ ]

🌟 Outcome

🗒️ Template - Meeting notes

🗓️ Date

👥 Participants

-
-

🥅 Goals

-

🗣️ Discussion topics

Time	Item	Presenter	Notes
			•

✅ Action items

- [ ]

⤴️ Decisions

↰

Understanding AI Models for Privileged Access Management (PAM) & User Footprint Knowledge Graph

---

1. Overview

To build an **AI-powered PAM system**, we need three key AI models working together:

1️⃣ **Graph Neural Networks (GNNs)** → **Detect privilege abuse paths** in Neo4j.\
2️⃣ **Autoencoders** → **Identify anomalous access behavior** (unsupervised learning).\
3️⃣ **Llama (LLM-based chatbot)** → **Explain risk patterns & answer SOC analyst queries**.

Each model plays a distinct role in making PAM predictive, explainable, and proactive.

---

2. What Are These AI Models?

Graph Neural Networks (GNNs)

- **What It Is:** Deep learning model designed for graph-structured data.
- **Use in PAM:** Learns user-role-privilege relationships & detects **hidden privilege escalations**.
- **Example Use Case:** If a user **gains admin access indirectly via multiple roles**, GNN detects it.
- **Tech Stack: Neo4j + PyTorch Geometric**.

Autoencoders (Unsupervised ML)

- **What It Is:** A neural network trained to reconstruct normal data and flag anomalies.
- **Use in PAM:** Detects **unusual user access behavior** based on historical patterns.
- **Example Use Case:** A dormant admin account is suddenly **used after 6 months** → flagged.
- **Tech Stack: TensorFlow/Keras Autoencoder.**

🚀 Sample Implementation: TensorFlow/Keras Autoencoder for PAM

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# Generate synthetic access log data
np.random.seed(42)
data = np.random.rand(10000, 20)  # 10,000 samples, 20 features

# Define the autoencoder model
input_dim = data.shape[1]
encoding_dim = 8  # Compressed representation

input_layer = keras.Input(shape=(input_dim,))
encoded = layers.Dense(encoding_dim, activation='relu')(input_layer)
decoded = layers.Dense(input_dim, activation='sigmoid')(encoded)

autoencoder = keras.Model(input_layer, decoded)
autoencoder.compile(optimizer='adam', loss='mse')

```

```python
# Train the model
autoencoder.fit(data, data, epochs=50, batch_size=32, shuffle=True)

# Use the trained model for anomaly detection
reconstructed = autoencoder.predict(data)
anomaly_score = np.mean(np.abs(data - reconstructed), axis=1)
threshold = np.percentile(anomaly_score, 95)  # Set a threshold
anomalies = data[anomaly_score > threshold]
print(f"Detected {len(anomalies)} anomalies")
```

Llama (LLM-based SOC Chatbot)

- **What It Is:** A large language model (LLM) optimized for security queries.
- **Use in PAM:** Allows SOC analysts to ask questions like **"Who has excessive privileges?"**.
- **Example Use Case:** If Alice is flagged, the SOC analyst asks **"Why is Alice high risk?"**, and Llama retrieves an explanation.
- **Tech Stack: Llama 3, Mistral 7B + Pinecone for RAG-based retrieval**.

---

3. How These Models Work Together

Model	Function	Example Query
GNNs	Find hidden privilege paths	"Does Alice have an indirect admin role?"
Autoencoders	Detect access anomalies	"Which users suddenly escalated privileges?"
Llama (LLM)	Explain risk insights	"Why is Alice flagged as high risk?"

🚀 Integration Flow

1️⃣ **GNNs in Neo4j** detect privilege escalation paths.\
2️⃣ **Autoencoders in AWS Lambda** flag abnormal user access.\
3️⃣ **Llama (LLM) via API Gateway** answers SOC security questions.

---

4. Deployment Architecture

Component	Technology
Graph DB	Neo4j
Vector DB	Pinecone (stores security insights)
Anomaly Detection	TensorFlow/Keras Autoencoders
LLM-based Chatbot	Llama 3, Mistral 7B (via API)
API Backend	AWS Lambda (Python)
Event Processing	AWS EventBridge

---

5. Next Steps

1️⃣ **Build Python pipelines** for GNNs & Autoencoders.\
2️⃣ **Develop API endpoints** for SOC chatbot (Llama + Pinecone).\
3️⃣ **Design real-time anomaly detection workflows** with EventBridge.

🚀 **Which part should we start implementing first?**

Developing Graph-Based Risk Analysis Using Graph Neural Networks (GNNs) and Neo4j

1️⃣ Key Steps to Build Graph-Based Risk Analysis

1️⃣ **Build the Knowledge Graph in Neo4j** → Store user-role-privilege relationships.
2️⃣ **Extract Graph Features for GNN Training** → Convert graph data into numerical representations.
3️⃣ **Train a GNN Model on Privilege Escalation Data** → Identify risky user access paths.
4️⃣ **Deploy GNN Inference for Risk Prediction** → Real-time risk scoring for new privilege assignments.
5️⃣ **Integrate with APIs for SOC Analysis** → Expose AI-driven risk scores via API.

2️⃣ Step-by-Step Guide to Implementing GNN with Neo4j

Step 1: Create a Graph in Neo4j

📌 Graph Schema

```
(User) → [:HAS_ROLE] → (Role)
(Role) → [:GRANTS_ACCESS] → (Privilege)
(User) → [:PERFORMED_ACTION] → (Resource)
```

This structure helps detect **who has access to what and how privileges are inherited.**

🔹 Example Neo4j Cypher Query: Insert Data

```
CREATE (u1:User {name: "Alice", id: 1}),
       (u2:User {name: "Bob", id: 2}),
       (r1:Role {name: "Admin"}),
       (r2:Role {name: "Developer"}),
       (p1:Privilege {name: "Access AWS"}),
       (p2:Privilege {name: "Modify Databases"})

MERGE (u1)-[:HAS_ROLE]->(r1)
MERGE (r1)-[:GRANTS_ACCESS]->(p1)
MERGE (u2)-[:HAS_ROLE]->(r2)
MERGE (r2)-[:GRANTS_ACCESS]->(p2);
```

Step 2: Extract Graph Features for GNN Training

🔹 Generate Node Embeddings in Neo4j

```
CALL gds.fastRP.write({
  nodeProjection: 'User',
  embeddingDimension: 128,
  writeProperty: 'embedding'
})
```

This saves **128-dimensional vectors** for each node, representing user privilege relationships.

Step 3: Train a GNN Model to Detect Risk

📌 Install Dependencies

```
pip install torch torch-geometric torch-scatter neo4j
```

🔹 Python Code to Train a GNN Model

```python
import torch
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from neo4j import GraphDatabase

# Define the GNN Model
class RiskGNN(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super(RiskGNN, self).__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = self.conv2(x, edge_index)
        return F.softmax(x, dim=1)
```

Step 4: Deploy the GNN for Risk Prediction

🔹 API Endpoint for Real-Time Risk Prediction

```python
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route("/predict_risk", methods=["POST"])
def predict_risk():
    user_id = request.json["user_id"]
    risk_score = model(torch.tensor([embedding]))[0].tolist()
    return jsonify({"user_id": user_id, "risk_score": risk_score})

if __name__ == "__main__":
    app.run(port=5000)
```

Step 5: Integrate with SOC Analyst Queries

Query	Response from LLM (Llama 3)
“Who has hidden admin privileges?”	Alice, via Helpdesk role inheritance.
“Is Bob a risk?”	Bob has never used his Admin role, high risk.
“Show high-risk access paths.”	Alice → Helpdesk → Admin (Escalation detected).

3️⃣ Summary: GNN-Based Risk Analysis Workflow

Step	Action	Tech Stack
1. Build Knowledge Graph	Store users, roles, and privileges	Neo4j
2. Generate Graph Features	Compute embeddings (Node2Vec)	Neo4j Graph Data Science
3. Train GNN Model	Detect privilege risks	PyTorch Geometric
4. Deploy GNN Inference	Real-time risk predictions	Flask API
5. Integrate SOC Chatbot	LLM explains risks	Llama 3 (API)

REST API vs. GraphQL for Frontend

Since the **TrustX Adaptive API** will primarily serve the **frontend (hosted on Cloudflare)**, we need to choose between **REST API and GraphQL** based on:

- **Performance**
- **Data Querying Flexibility**
- **Security**
- **Scalability**

1️⃣ Comparing REST API vs. GraphQL

Criteria	REST API	GraphQL
Data Fetching	Over-fetching (fixed endpoints return full response)	Exact fetching (only requested fields)
Performance	Can lead to multiple API calls	Optimized single API call for multiple data points
Frontend Flexibility	Limited to predefined responses	Flexible queries (UI requests only what it needs)
Caching	Easy to cache via Cloudflare, AWS API Gateway	Complex caching, requires GraphQL-aware cache
Authentication & Security	Simple, can use JWT & API keys	More complex, requires field-level access control
Learning Curve	Well known, easier to integrate	Requires GraphQL-aware frontend & dev expertise
Error Handling	Traditional HTTP status codes	Requires proper schema error handling
File Uploads (S3, Secrets Manager)	Easier	Requires additional setup

---

2️⃣ Which is Better for TrustX Adaptive API?

Use REST API If:

✅ Cloudflare caching is critical for performance.
✅ API requests are mostly structured and follow predefined workflows.
✅ Security concerns make field-level access control complex in GraphQL.
✅ You need easier integration with AWS Lambda, API Gateway, and IAM.

Use GraphQL If:

✅ The frontend needs flexible data fetching and fewer API calls.
✅ The UI frequently **combines multiple API calls** into one request.
✅ You want to optimize network performance by reducing over-fetching.
✅ Security policies allow managing **granular field access control**.

3️⃣ Final Recommendation: Hybrid Approach

- **Use REST API** for most endpoints where caching & security matter.
- **Use GraphQL** for **user profile, knowledge graph querying, and AI-driven insights**.

🔹 **Example API Design:**

```
1 bash
```

CopyEdit

```
https://api.trustx.cloud/ ├── /auth/login          (REST - User Authentication) ├── /users/profile       (GraphQL -
Flexible user data query) ├── /users/access       (GraphQL - Fetch knowledge graph-based permissions) ├──
/integrations/status (REST - Simple JSON status response) ├── /risk/assessments    (GraphQL - AI-driven analysis &
recommendations) ├── /audit/logs          (REST - Compliance logs API)
```

4️⃣ Deployment Considerations

✅ **Cloudflare + REST** → API Gateway caching works best.
✅ **Cloudflare + GraphQL** → Requires **custom GraphQL caching** (Apollo, Cloudflare Workers).
✅ **Security** → REST is simpler for **Auth0 + AWS IAM-based authentication**.

TrustX Adaptive API - GraphQL Schema for Knowledge Graph

1. Introduction

The **TrustX GraphQL API** enables flexible querying of **user access footprints, privilege relationships, and AI-driven insights**. This API supports **frontend applications** hosted on Cloudflare and provides **structured, optimized data retrieval** from the **User Footprint Knowledge Graph**.

GraphQL Endpoint

```
<https://api.trustx.cloud/graphql>
```

---

2. GraphQL Schema Design

This schema defines **Users, Roles, Applications, Access Logs, and AI Insights**.

🔹 User & Privilege Graph

```graphql
type User {
  id: ID!
  username: String!
  email: String!
  roles: [Role!]!
  permissions: [String!]!
  riskScore: Float
  lastActivity: String
}

type Role {
  id: ID!
  name: String!
  permissions: [String!]!
  users: [User!]!
}

type Application {
  id: ID!
  name: String!
  owner: User
  roles: [Role!]!
  riskLevel: String
}

type AccessLog {
  id: ID!
  user: User!
  action: String!
  timestamp: String!
}

type AnomalyDetection {
  id: ID!
  user: User!
  description: String!
```

```
  severity: String!
  detectedAt: String!
}
```

🔹 Query API

```graphql
type Query {
  getUser(id: ID!): User
  getUsersByRole(roleId: ID!): [User!]
  getUserAccessLogs(userId: ID!): [AccessLog!]
  getApplications: [Application!]
  getRiskyUsers(threshold: Float!): [User!]
  getAnomalies: [AnomalyDetection!]
}
```

🔹 Mutation API

```graphql
type Mutation {
  updateUserRole(userId: ID!, roleId: ID!): User
  revokePermission(userId: ID!, permission: String!): User
  createApplication(name: String!, ownerId: ID!): Application
  logAccess(userId: ID!, action: String!): AccessLog
}
```

---

3. API Security & Authentication

✅ **Auth0 OAuth 2.0** → Secures GraphQL API with JWT tokens.
✅ **Role-Based Access Control (RBAC)** → Users can only query permitted data.
✅ **Rate Limiting & Caching** → Optimized for Cloudflare GraphQL caching.
✅ **Field-Level Access Control** → Ensures **least privilege** queries.

---

4. Next Steps

- **Implement API Resolvers** for fetching data from **Neo4j (GraphDB)**.
- **Integrate AI Risk Engine** for anomaly detection queries.
- **Develop Frontend GraphQL Queries** for TrustX UI.
- **Finalize Hybrid API Proposal** (REST + GraphQL structure).